FM School

Project guide for the SEPM lab

Run the application, follow the code, and explain the tests in the viva. GitHub repository

1. Set up and run
2. Understand the code
3. Demo the main workflows
4. Use the JSON API
5. Know the nine new tests
6. Run CI and make changes
7. Prepare for the viva

1 Set up and run

FM School manages Teacher and Student accounts, profiles, course requests, and marks. The Admin manages records. Teachers and Students work with their own profiles and enrollments. The project uses Java 25, Spring Boot, Maven, and PostgreSQL 18.6.

1.1 Start on a Windows lab PC

Install Git, JDK 25, Docker Desktop, and IntelliJ. Start Docker Desktop and wait for its engine. Select JDK 25 for the IntelliJ project and Maven importer. For terminal commands, set `JAVA_HOME` to the JDK directory and check `java -version` and `.\mvnw.cmd -version`.

```
git clone https://github.com/FahimHossen121/fm-school-academic.git
cd fm-school-academic
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\run-demo.ps1
```

The runner starts PostgreSQL and Spring Boot, applies the demo seed after startup, and opens localhost:8080/login. The seed resets the local Admin password to the demo value. Use it on a lab database.

Role	Username / password	First page
Admin	`admin` / `12345678`	Admin home
Teacher	`teacher2` / `teacher2`	My Profile
Student	`student2` / `student2`	My Profile

1.2 Run the steps manually if needed

Start the database with `docker compose up -d postgres`. In IntelliJ, run `FmSchoolApplication` and wait for Liquibase to finish. Load the demo rows from a second terminal and refresh the login page.

```
Get-Content .\demo\seed-demo-data.sql |
  docker compose exec -T postgres psql -v ON_ERROR_STOP=1 -U myuser -d fm_school_db
```

Stop Spring Boot with the IDE Stop button if you started it there. If you used the runner, identify its Java process before stopping it. `docker compose stop` stops PostgreSQL and keeps the volume. GitHub stores code and migrations. Use the seed to recreate demo data on another PC.

2 Understand the code

A web request enters Spring Security, which checks the user's identity and route permissions. The controller reads the request. The service applies business rules. The repository reads or writes the database. The controller returns an HTML view or JSON.



2.1 Find the right file

Java classes use the package `com.example.fmschool`. In IntelliJ, open files by name with Ctrl+Shift+N.

File or package	Responsibility
`SecurityConfig.java`	Form login, API authentication, role rules, and redirects.
`StudentController.java`	Admin Student CRUD endpoints and HTTP responses.
`EnrollmentApiController.java`	API requests to apply, invite, respond, remove, and edit.
`EnrollmentService.java`	Recipient checks, one accepted course, and Teacher edit authority.
`ProfileService.java`	Own profile, Admin edits, and account/profile deletion.
`PeopleApiService.java`	Admin account/profile changes and JSON response DTOs.
`entity/` and `repository/`	Java representations of stored data and the interfaces that query them.
`dto/`	Data shapes for requests and responses. Validation checks incoming fields.
`templates/`	Thymeleaf HTML views under `src/main/resources/`.

2.2 The four main tables

- **Accounts** hold the username, role, and password hash.
- **Students** hold a name and roll number. **Teachers** hold a name and one course. Their account links connect profiles to login identities.
- **Enrollments** connect a Student and Teacher. They record who requested the link, its status, and optional marks. One Student can have several pending applications and at most one accepted course.

Liquibase creates and updates the schema. Its changelog defines the one-accepted-course constraint and the foreign-key rules for enrollment deletion. Hibernate validates the schema and maps entities to tables.

2.3 Follow a profile update

When a Student submits My Profile, the controller validates the form and calls `ProfileService`. The service looks up the signed-in account, then finds and saves that account's Student profile. The name and roll belong to the profile. The username and password hash belong to the account.

3 Demo the main workflows

Role	Typical actions
Admin	Manage Teacher and Student accounts. Directly associate a Student with a Teacher and manage requests.
Teacher	Edit own profile, invite Students, respond to applications, and edit names, rolls, and marks for accepted Students.
Student	Edit own profile, apply to Teachers, respond to invitations, and view own marks.

3.1 A quick lab demonstration

1. Sign in as `teacher2`. Find an accepted Student in the enrollment board, open the edit page, and enter a mark such as 85.50.
2. Sign out and sign in as that Student. Check the mark on the enrollment board. Open My Profile to show how the signed-in identity selects the record.
3. To show a pending request, use a new Student and Teacher. Complete both profiles, then apply as the Student. Sign in as the Teacher and accept. You can also send an invitation as the Teacher and respond as the Student.
4. Sign in as Admin and open Manage Teachers or Manage Students. Show the management actions across accounts and profiles.

The seed already enrolls the demo Students. Use a fresh Student account when demonstrating an application or invitation.

3.2 Rules to check while demonstrating

- The recipient or Admin can accept or reject a pending request. A Teacher gains edit permission when their enrollment with the Student is accepted.
- A Student must leave the accepted course before accepting another. Removing an association leaves both accounts usable.
- Marks range from 0 to 100 with up to two decimal places. An empty mark means ungraded. A mark of zero is a grade.
- A Teacher can edit an accepted Student only through their own enrollment. Use another Teacher account to demonstrate the denial.
- Deleting a Teacher account removes the profile and its enrollment links. The affected Student accounts survive. Use disposable accounts for the demo.

3.3 Troubleshoot the demo

Symptom	First check
Database connection error	Run `docker compose ps`. Check Docker Desktop and the port mapping in `compose.yaml`.
Wrong Java version	Check terminal `java -version` and IntelliJ's Maven JDK.
Demo password rejected	Confirm the app started, then run the demo seed against the local lab database.
Port 8080 in use	Identify the process using the port. The runner and IntelliJ can start separate Java processes.
Styles look old	Rebuild and refresh. Rebuild Tailwind when changing its input or template classes.

4 Use the JSON API

The HTML controllers render Thymeleaf pages. The API controllers return JSON using the same services and ownership rules. A DTO carries selected fields between the app and the client. Response DTOs leave password hashes out of JSON.

4.1 Authentication and writes

Use your browser session or HTTP Basic authentication in Postman. For POST, PUT, and DELETE, first call `GET /api/csrf`. Keep the session cookie and send the returned token using the returned `headerName`. CSRF protection verifies that a browser write came from a client that obtained the token for that session.

Endpoint	Purpose
`GET /api/session`	See the authenticated username and roles.
`GET /api/csrf`	Obtain the token and header name.
`GET`, `PUT`, `DELETE /api/me`	View, edit, or remove your own Teacher or Student profile and login as applicable.
`/api/students` and `/api/teachers`	Admin lists and creates people. Add an ID to read, update, or delete one record.
`GET /api/enrollments`	Read the role-filtered board.
`POST /api/enrollments/applications`	Student applies with a `teacherId`.
`POST /api/enrollments/invitations`	Teacher invites with a `studentId`.
`POST /api/enrollments/associations`	Admin associates a Teacher and Student by profile IDs.
`POST /api/enrollments/{id}/accept`	Recipient or Admin accepts. Use `/reject` to decline.
`DELETE /api/enrollments/{id}`	Participant removes an accepted link. Admin can remove any link.
`PUT /api/enrollments/{id}/student`	Associated Teacher or Admin edits Student details and marks on an accepted link.

4.2 One request to show

As the associated Teacher, send this JSON to `PUT /api/enrollments/{id}/student`. The enrollment ID identifies the relationship. You can find it on the enrollment board.

```
{"name":"Student Two","roll":"9000002","marks":85.50}
```

A successful write returns 200. Repeat the call as an unrelated Teacher and expect 403. When accepting a second course, expect 409 and check that the original enrollment still exists.

Status codes. 200 success, 201 created, 204 success without a body, 400 invalid input, 401 authentication required, 403 permission or CSRF denied, 404 missing record, and 409 conflict with the current state.

5 Know the nine new tests

A unit test isolates one piece of logic. These six tests call `EnrollmentService` directly and use Mockito repositories. An integration test checks several parts working together. The three new integration tests use the full Spring application, MockMvc, Liquibase, and a separate PostgreSQL database. They read the stored results after HTTP requests complete.

5.1 Six unit tests

	Scenario	What it proves
U1	Accept twice	The retry returns 409. The record stays accepted and no second save runs.
U2	Reverse a rejected request	A Teacher invites a Student whose application they rejected. The same row reopens and the Student becomes the recipient.
U3	Filter the invitation list	Students who already have a course or lack a linked account stay out of the picker.
U4	Keep pending history	An enrolled Student still sees pending rows, while the board offers no new Teachers to apply to.
U5	Clear an existing mark	Clearing 78.25 saves a null mark, which means ungraded.
U6	Invite without a profile	A Teacher login without a profile gets a conflict and no enrollment is written.

5.2 Three integration tests

1. **Transfer courses.** Apply to two Teachers. Accept the first request, then attempt to accept the second. Check for 409 and confirm the second request stays pending in PostgreSQL. Remove the first enrollment, accept the second, and verify there is exactly one accepted row for the Student.
2. **Roll back a failed creation.** Ask Admin to create a Teacher with valid account details and no course. The profile step returns 400. Look up the username in PostgreSQL and verify the earlier account insert was rolled back.
3. **Delete a Teacher without losing a Student.** Create an accepted enrollment, delete the Teacher as Admin, and query the database. The Teacher account, profile, and enrollment are gone. The Student profile survives and the Student can still sign in with the original credentials.

The existing application startup check also runs in the integration phase.

5.3 Explain the rollback test in the viva

The HTTP response tells the client the course is required. Querying the accounts table shows that the earlier insert was rolled back. A response assertion alone could pass even if an unused account were left in the database.

6 Run CI and make changes

6.1 Run tests on your PC

Run unit and isolated web tests with `.\mvnw.cmd test`. To run the integration tests too, start a disposable PostgreSQL container on port 55432. The demo database uses port 5432, so both can run independently.

```
docker run -d --rm --name fm-school-lab-test ‘
  -p 127.0.0.1:55432:5432 ‘
  -e POSTGRES_DB=fm_school_test -e POSTGRES_USER=fm_school_test ‘
  -e POSTGRES_PASSWORD=fm_school_test postgres:18.6
docker exec fm-school-lab-test pg_isready -U fm_school_test -d fm_school_test
```

Once `pg_isready` reports that the database accepts connections, run the suite and stop the disposable container when finished.

```
.\mvnw.cmd clean verify
docker stop fm-school-lab-test
```

The integration profile points to this dedicated database. Maven Surefire runs `*Tests`. Maven Failsafe runs `*IT` during `verify`. Reports are in `target/surefire-reports` and `target/failsafe-reports`.

6.2 What GitHub Actions does

The workflow is `.github/workflows/tests.yml`. A push or pull request starts an Ubuntu job. GitHub starts a clean PostgreSQL service, waits for its health check, sets up Java 25, and executes `mvn verify`. Open the Actions tab, select FM School tests, then open the `verify` job to see the test totals and result.

6.3 Make a small requested change

Trace the feature from controller to service before editing. For a marks range change, check the form, input validation, database constraint, and boundary tests. Add a new Liquibase changeset when the stored schema changes.

Change	Start here
Page text or layout	Thymeleaf files in `src/main/resources/templates/`.
Colors and spacing	Tailwind source in `src/main/resources/styles/`. Run `npm.cmd ci`, then `npm.cmd run css:build`. Commit generated CSS with the source change.
Permission or workflow	`SecurityConfig` for route rules. The relevant service for record ownership and business rules.
Database column	Entity and a new Liquibase changeset. Update DTOs, forms, and tests as needed.

Pull changes before working on a different PC. After editing, run the tests and review `git diff`. Commit the specific files, push, and check the new Actions run.

7 Prepare for the viva

7.1 How did you create the REST API?

Open `StudentController.java`. Show `@RestController`, `@RequestMapping("/api/students")`, and `@GetMapping`. The GET method delegates to `PeopleApiService.students()`, which queries the repository and maps results to response DTOs. In the POST method, `@RequestBody` binds JSON, `@Valid` checks the request, and creation returns 201 with a Location header.

7.2 How did you use Docker?

Open `compose.yaml`. PostgreSQL 18.6 runs in Docker, and Spring Boot runs on the host through IntelliJ or Maven. Compose sets the database name and login, publishes 5432 to localhost, and attaches `postgres_data` to preserve data across ordinary container recreation. Show `docker compose ps`. Explain that the PostgreSQL driver connects Java to the database.

7.3 How did you implement Spring Data JPA?

Show `Student.java`, `StudentRepository.java`, then `StudentService.updateStudent()`. The entity maps Java fields to columns. The repository provides methods such as `findById` and `save`. The service loads a Student, changes its fields, and saves within a transaction. Hibernate handles the mapping. Liquibase manages changes to the schema.

7.4 How did you implement Spring Security?

Open `SecurityConfig.java`. Show the encoder, account lookup through `UserDetailsService`, and the web and API filter chains. A user first proves who they are. Route rules decide what their role may access. Open `EnrollmentService.requireEditor()` for the record-level check. A Teacher can edit the Student when the accepted enrollment belongs to that Teacher's account. The server applies this rule to direct API requests too.

7.5 What distinguishes the two test types?

In `EnrollmentServiceUnitTests.java`, Mockito supplies the repository responses and the test checks the service logic. In `EnrollmentPersistenceIT.java`, a request passes through security, controller, service, and real database transactions. Show how the rollback test checks both the response and the absence of the account in PostgreSQL. Finish by showing the Actions run.

7.6 Original material

- Full project guide and detailed viva notes
- Windows lab setup and feature walkthroughs
- JSON API reference and Tailwind workflow
- REST API, Spring Boot, and Docker lab report